

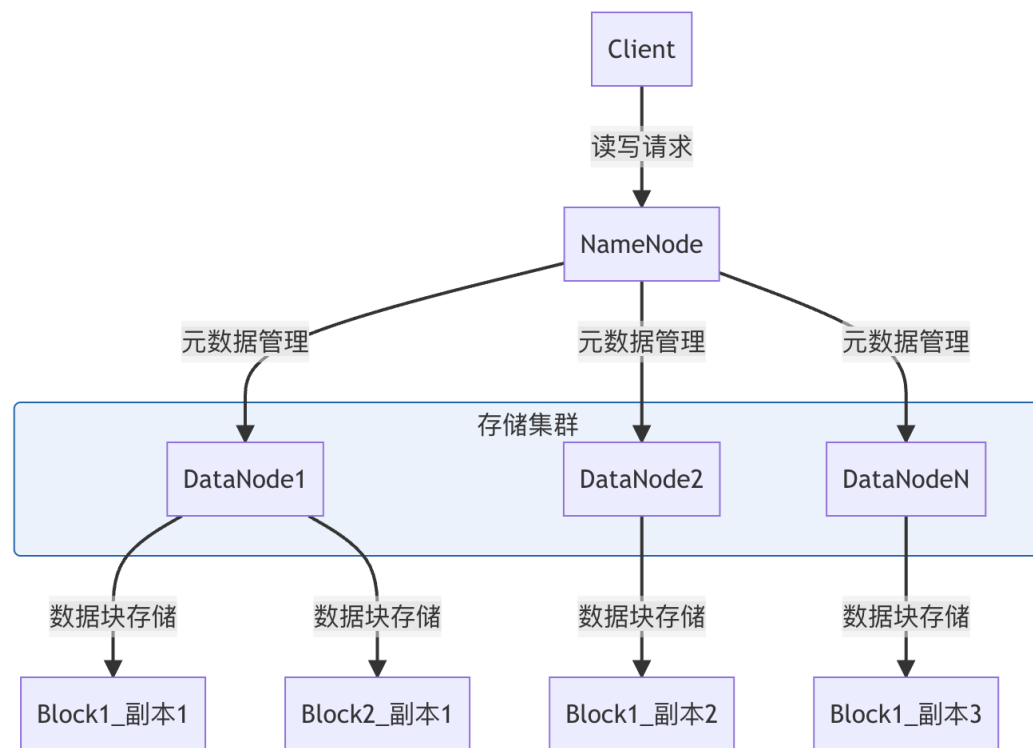
Hadoop和Spark部署教程

大数据平台技术

原理部分——分布式存储管理

单节点存储面临的挑战：在全球数据量以年均40%速度增长的背景下，传统集中式存储（如SAN、NAS）面临**单节点容量上限、读写性能瓶颈、故障单点风险**等挑战。

分布式存储（Distributed Storage）：将数据分散存储在**多台独立物理设备**上，通过协同协议实现**数据冗余、负载均衡与高可用**。



关键组件说明：

1. **NameNode**：管理元数据（文件目录、块位置），不存储实际数据；
2. **DataNode**：存储数据块，定期向NameNode汇报块信息；
3. **Client**：通过NameNode获取块位置，直接与DataNode读写数据。

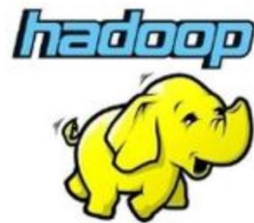
HDFS 读写流程：

写数据：

1. Client → NameNode（问存哪）
2. NameNode → 返回 DataNode 列表
3. Client → DataNode（写数据 + 副本复制）

读数据：

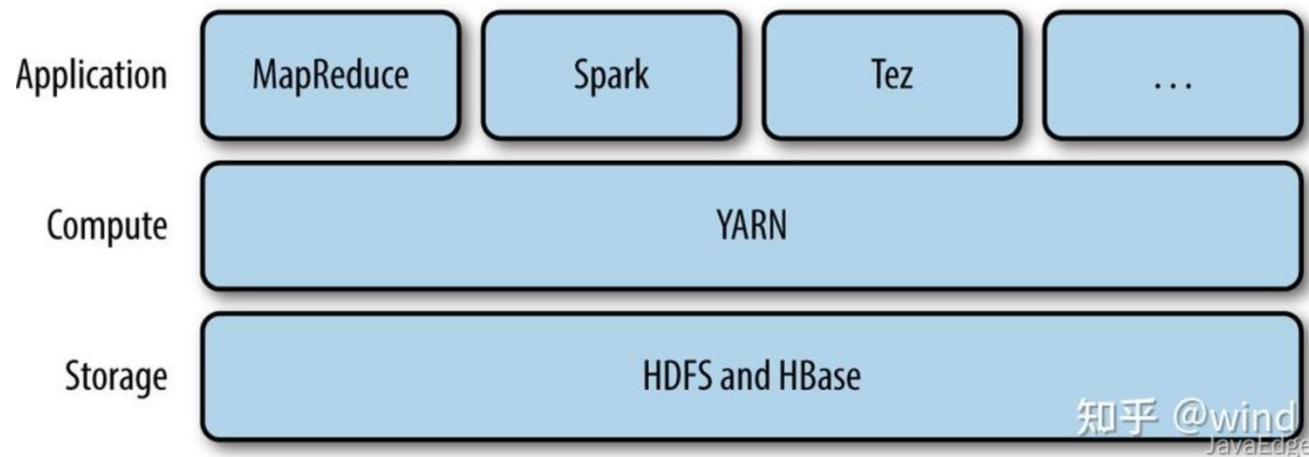
1. Client → NameNode（问在哪）
2. NameNode → 返回位置
3. Client → DataNode（直接读）



原理部分——Hadoop介绍

Hadoop是什么？

Apache Hadoop 是一个集**存储（HDFS）**、**计算（MapReduce）**和**资源调度（YARN）**于一身的海量数据处理平台。它允许我们用大量廉价的普通服务器，去可靠地存储和处理单个超大规模计算机无法完成的任务。



Hadoop核心应用场景？

Hadoop 的核心定位是海量数据的批量存储与离线处理。

1. 机器学习数据预处理
2. 数据归档与合规存储
3. 海量日志存储与分析

Hadoop干什么？

1. 存数据：HDFS（分布式文件系统）

核心思想：文件太大，一台机器存不下？那就切分成多个数据块（默认128MB），并分别存储在由普通服务器组成的大集群中。

特点：高容错（自动保存多个副本，通常3份），高吞吐量（适合大文件的一次写入、多次读取）。

2. 资源调度：YARN(资源调度平台)

统一管理集群中所有服务器的CPU和内存，决定哪个任务（比如一个MapReduce程序或一个Spark应用）可以使用多少资源、在哪个服务器上运行。

3. 算数据：MapReduce、Spark等计算引擎

运行具体的计算逻辑，处理HDFS上的海量数据。

MapReduce：Hadoop自带的批处理引擎，稳重但偏慢（磁盘级计算）。

Spark：更快的内存级计算引擎，需要跑在YARN上（或其他资源管理器上）。

原理部分——Spark介绍



Spark是什么？

Apache Spark 是一个快速的、通用的、内存级**计算引擎**，用于处理大规模数据；是 Hadoop MapReduce 的"升级版"——算得更快、写得更简单、功能更全面。Spark 不是存储系统，它只负责"算"，不负责"存"（通常搭配 HDFS 或 S3 使用）。

Spark vs MapReduce

MapReduce	Spark
数据存储结构： 磁盘 HDFS文件系统的split	使用 内存 构建弹性分布式数据集 RDD 对数据进行运算和cache
编程范式 ：Map+Reduce	编程范式： DAG ：Transformation + Action
计算中间结果落到磁盘，IO及序列化、反序列化代价大	计算中间结果在内存中维护，存取速度比磁盘高几个数量级
Task以 进程 的方式维护，需要数秒时间才能启动任务	Task以 线程 的方式维护，对于小数据集读取能达到更细的粒度

MapReduce 的计算模型（磁盘级）：

Step 1: 读磁盘 → 计算 → 写磁盘
Step 2: 读磁盘 → 计算 → 写磁盘
Step 3: 读磁盘 → 计算 → 输出
↑ 每个步骤都有磁盘I/O

Spark 的计算模型（内存级）：

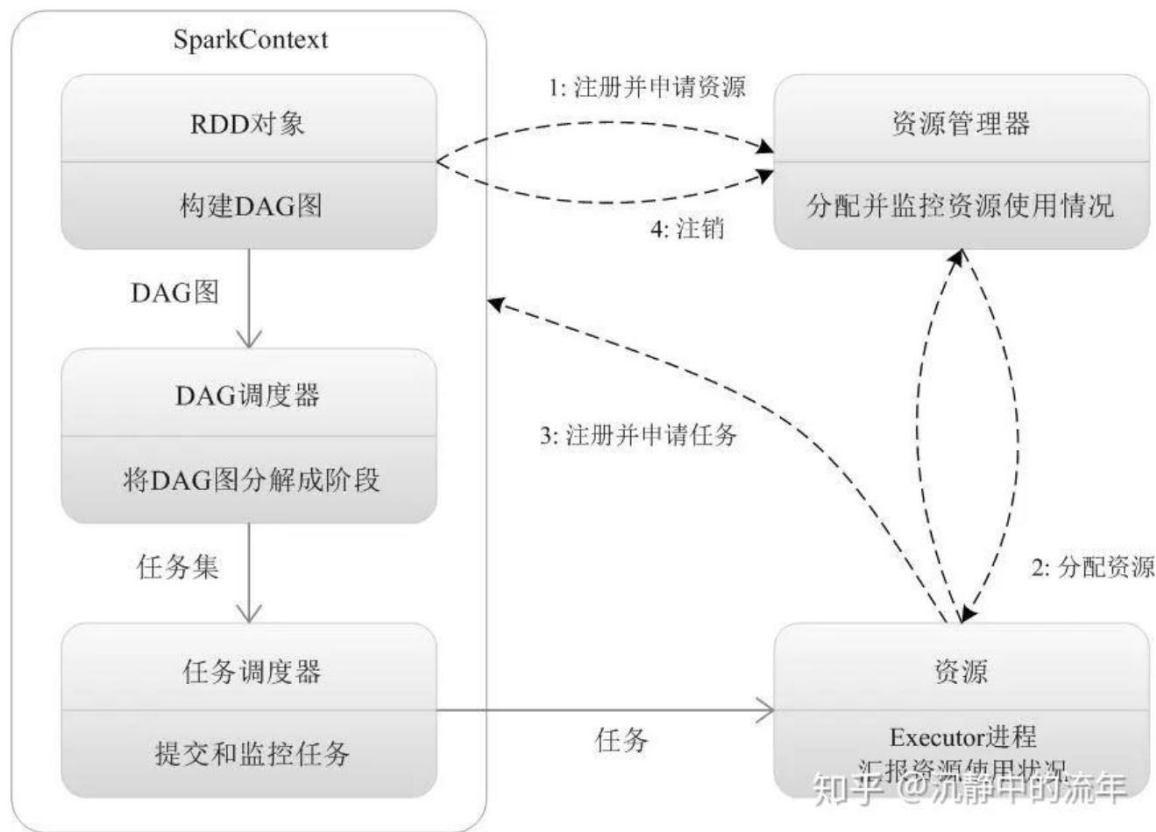
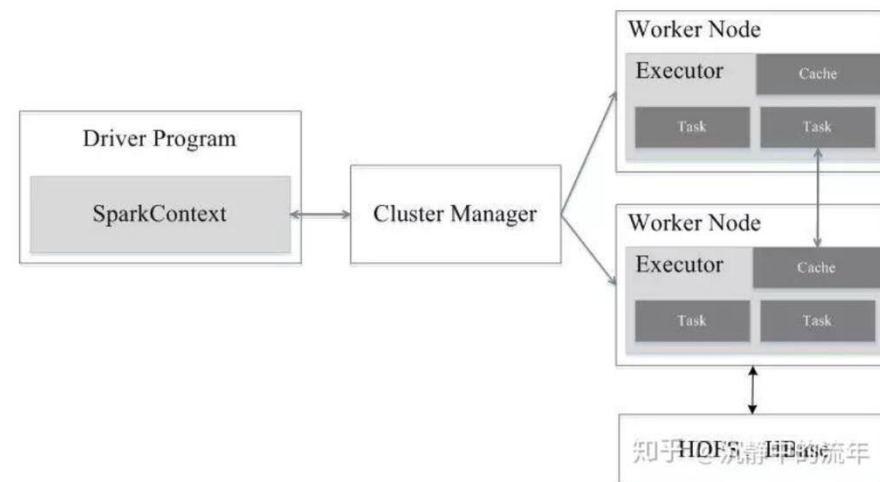
Step 1: 读磁盘 → 计算 → 存入内存
Step 2: 从内存读 → 计算 → 存入内存
Step 3: 从内存读 → 计算 → 输出
↑ 中间结果在内存中流通

- ◎ **RDD**：是弹性分布式数据集（Resilient Distributed Dataset）的简称，是分布式内存的一个抽象概念，提供了一种高度受限的共享内存模型。
- ◎ **DAG**：是Directed Acyclic Graph（有向无环图）的简称，反映RDD之间的依赖关系。

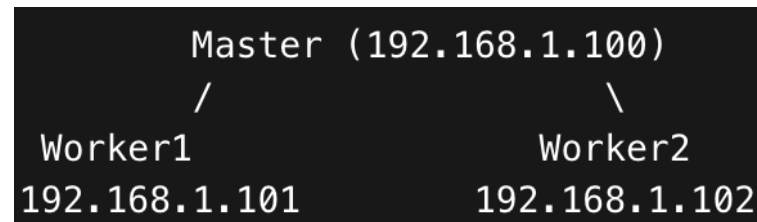
原理部分——Spark介绍

Spark 工作流程:

- 1, Application首先被Driver构建DAG图并分解成Stage。
- 2, 然后Driver向Cluster Manager申请资源。
- 3, Cluster Manager向某些Work Node发送征召信号。
- 4, 被征召的Work Node启动Executor进程响应征召, 并向Driver申请任务。
- 5, Driver分配Task给Work Node。
- 6, Executor以Stage为单位执行Task, 期间Driver进行监控。
- 7, Driver收到Executor任务完成的信号后向Cluster Manager发送注销信号。
- 8, Cluster Manager向Work Node发送释放资源信号。
- 9, Work Node对应Executor停止运行。



部署教程——Hadoop多机部署



环境准备

◎ 系统要求

- **操作系统**: Linux (Ubuntu/CentOS 等), 推荐使用 Ubuntu 20.04 及以上版本
- **Java 版本**: Java 8 或 Java 11 (Hadoop 3.x 要求 JDK 8 以上)
- **SSH**: 需要安装 SSH 并启动 sshd 服务

假设我们有三台服务器:

Master (主节点) : 192.168.1.100

Worker1 (从节点) : 192.168.1.101

Worker2 (从节点) : 192.168.1.102

三台机器配置相同 (CPU / 内存 / 系统)

这三台机器, 怎么变成一个“集群”?

1. 让三台机器能互相通信 (SSH)
2. 安装同样的软件 (Hadoop)
3. 告诉系统谁是 Master, 谁是 Worker
4. 启动整个集群

部署流程



1. 所有节点统一环境 (Java / 主机名 / hosts)
2. 配置 SSH 免密 (Master → Workers)
3. 安装并分发 Hadoop
4. 配置集群 (workers + XML)
5. 格式化并启动
6. 验证集群

Step1: 在所有机器上安装Java

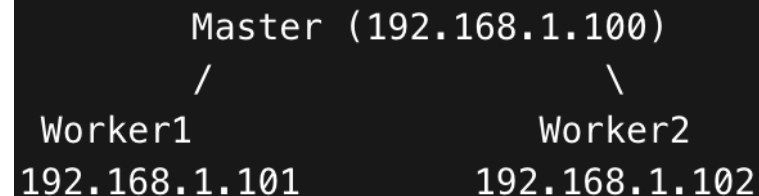
代码块

```
1  # Ubuntu/Debian 系统
2  sudo apt update
3  sudo apt install openjdk-8-jdk -y
4
5  # CentOS/RHEL 系统
6  sudo yum install java-1.8.0-openjdk-devel -y
```

代码块

```
1  java -version
2  # 应显示 openjdk version "1.8.0_xxx"
```

部署教程——Hadoop多机部署



Step2: 配置hosts(所有机器都要做)

让机器之间能“用名字通信”

代码块

```
1 sudo vim /etc/hosts
```

打开编辑器后，输入：

代码块

```
1 192.168.1.100 master
2 192.168.1.101 worker1
3 192.168.1.102 worker2
```

Step3:配置 SSH 免密 (只在 Master 做)

在master节点对应服务器上配置：

代码块

```
1 ssh-keygen -t rsa -P ""
2 ssh-copy-id worker1
3 ssh-copy-id worker2
```

测试是否配置成功：（如果不需要密码就能链接说明成功）

代码块

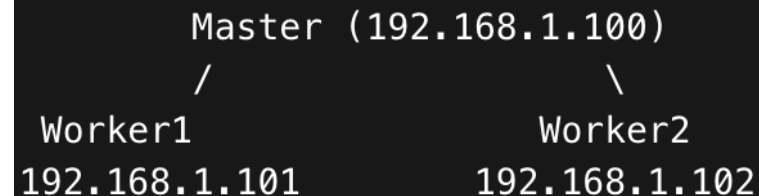
```
1 ssh worker1
2 ssh worker2
```

Step 4: 安装 Hadoop (只在 Master)

代码块

```
1 wget https://archive.apache.org/dist/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
2 tar -zxvf hadoop-3.3.6.tar.gz
3 mv hadoop-3.3.6 ~/hadoop
```

部署教程——Hadoop多机部署



Step 5: 分发 Hadoop (关键)

代码块

```
1 scp -r ~/hadoop worker1:~
2 scp -r ~/hadoop worker2:~
```

这样所有节点都有 Hadoop



Step 6: 配置环境变量 (所有机器)

在所有节点对应服务器上配置：

代码块

```
1 vim ~/.bashrc
```

添加：

代码块

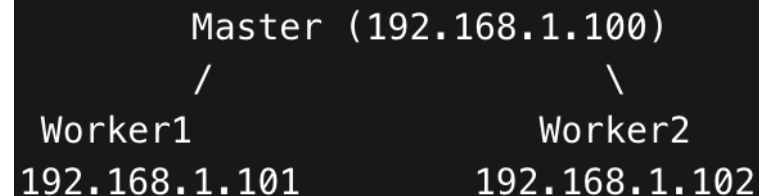
```
1 export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
2 export HADOOP_HOME=~/.hadoop
3 export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

激活：

代码块

```
1 source ~/.bashrc
```

部署教程——Hadoop多机部署



Step 7: 配置 Hadoop (只在 Master)

代码块

```
1 cd ~/hadoop/etc/hadoop
```

代码块

```
1 vim workers
```

现模)

代码块

```
1 worker1
2 worker2
```

2. 编辑core-site.xml

代码块

```
1 <configuration>
2   <property>
3     <name>fs.defaultFS</name>
4     <value>hdfs://master:9000</value>
5   </property>
6 </configuration>
```

3. 编辑hdfs-site.xml

代码块

```
1 <configuration>
2   <property>
3     <name>dfs.replication</name>
4     <value>2</value>
5   </property>
6 </configuration>
```

4. 编辑yarn-site.xml

代码块

```
1 <configuration>
2   <property>
3     <name>yarn.resourcemanager.hostname</name>
4     <value>master</value>
5   </property>
6 </configuration>
```

5. 编辑mapred-site.xml

代码块

```
1 cp mapred-site.xml.template mapred-site.xml
2
3 <configuration>
4   <property>
5     <name>mapreduce.framework.name</name>
6     <value>yarn</value>
7   </property>
8 </configuration>
```

部署教程——Hadoop多机部署

```
Master (192.168.1.100)
/
Worker1  Worker2
192.168.1.101 192.168.1.102
```

Step 8: 格式化 HDFS (只在

代码块

```
1 hdfs namenode -format
```

Step 9: 启动集群 (只在 Master)

代码块

```
1 start-dfs.sh
2 start-yarn.sh
```

Master 会自动 SSH 到 Worker 启动

Step 10: 验证是否成功

👉 Master 上:

代码块

```
1 jps
```

正确输出:

代码块

```
1 NameNode
2 ResourceManager
```

👉 Worker 上:

代码块

```
1 jps
```

正确输出:

代码块

```
1 DataNode
2 NodeManager
```

👉 Web UI

HDFS : <http://master:9870>
YARN : <http://master:8088>

Step 11: 简单测试

上传文件

代码块

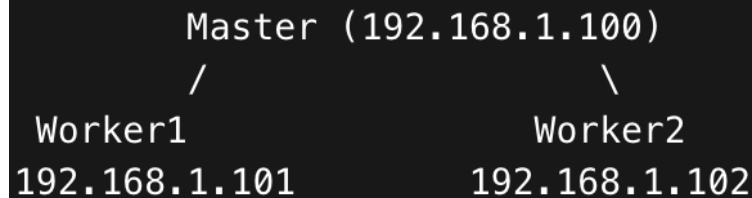
```
1 hdfs dfs -mkdir /input
2 echo "hello hadoop" > test.txt
3 hdfs dfs -put test.txt /input
```

查看文件 `hdfs dfs -ls /input`
应该能看见我们刚刚上传的文件

```
hdfs dfs -cat /input/test.txt
```

能看见文件的内容 "hello hadoop"

部署教程——Spark配置



Apache Spark 不负责管理机器，它借用 Hadoop 的 YARN 来调度资源

我们刚刚已经搭好了 Hadoop 集群，它已经具备两个能力：

1. HDFS：能存数据

2. YARN：能分配资源

那接下来我们要做的就是：

👉 **用 Spark 去“利用这些资源”来做计算**

Step 1: 下载 Spark (Master 上)

代码块

```
1 wget https://archive.apache.org/dist/spark/spark-3.5.1/spark-3.5.1-bin-hadoop3.tgz
2 tar -zxvf spark-3.5.1-bin-hadoop3.tgz
3 mv spark-3.5.1-bin-hadoop3 ~/spark
```

必须选择 带 hadoop 的版本 (bin-hadoop3)

Step 2: 配置环境变量 (所有节点)

vim ~/.bashrc **添加spark**

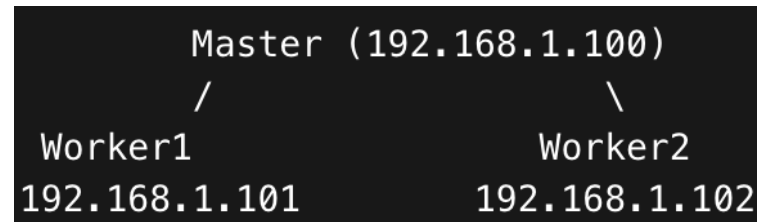
代码块

```
1 export SPARK_HOME=~/spark
2 export PATH=$PATH:$SPARK_HOME/bin
```

代码块

```
1 source ~/.bashrc
```

部署教程——Spark配置



Step 3: 配置 Spark 使用 YARN (核心)

代码块

```
1 cd ~/spark/conf
2 cp spark-env.sh.template spark-env.sh
```

编辑 vim spark-env.sh

代码块

添加

```
1 export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
```

这一行的作用是：让 Spark 知道 Hadoop 集群在哪

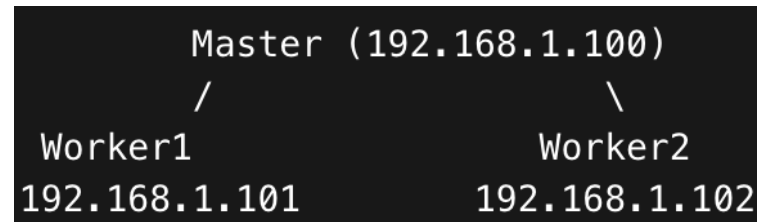


Step 4: 分发 Spark 到 Worker

代码块

```
1 scp -r ~/spark worker1:~
2 scp -r ~/spark worker2:~
```

部署教程——Spark测试



我们现在用 Spark, 在 Hadoop 集群上统计单词数量

Step1: 准备数据 (HDFS)

代码块

```
1 echo "hello hadoop spark hello" > test.txt
2 hdfs dfs -mkdir -p /spark-input
3 hdfs dfs -put -f test.txt /spark-input
```

数据已经放到 HDFS 里了 (分布式存储)

Step 4: 查看结果

```
hdfs dfs -cat /spark-output/part-00000
```

输出

代码块

```
1 hello 2
2 hadoop 1
3 spark 1
```

说明部署成功!

Step 2: 提交 Spark 任务 (核心)

代码块

```
1 spark-submit \
2 --master yarn \
3 --deploy-mode client \
4 $SPARK_HOME/examples/jars/spark-examples_*.jar \
5 wordcount \
6 hdfs://master:9000/spark-input \
7 hdfs://master:9000/spark-output
```

使用hadoop集群

输入文件

输出文件

wordcount是spark自带的示例程序, 安装时自带